

ALEX H-Model 自動量化交易系統開發規格計畫書

GitHub Codespaces & GitHub Actions + Gate.io 全自動部署規範

系統架構：Python 3.10 / CCXT / GitHub Actions

目標交易所：Gate.io (Futures / Swap)

執行頻率：Hourly Scheduled Cron (1h)

文件版本：v1.0.0 (Production-Ready)

一、專案概述與開發目標

本計畫書旨在引導 AI Agent (GitHub Copilot / Cursor / Claude Code) 在 **GitHub Codespaces** 環境中自動構建並完成 **ALEX H-Model 價差策略自動化交易系統**。本系統實現無人值守運作，透過 GitHub Actions 排程監控 Gate.io 現貨與永續合約價差 (Spread) 及日線偏離率 (BIAS%)，於條件滿足時自動執行訂單下單與風險控制。

二、系統核心架構與目錄結構

專案目錄設計遵循極簡且具擴充性的原則，Agent 在建立 Repo 時需完整生成以下目錄結構：

```
.
├── .devcontainer/
│   └── devcontainer.json          # Codespaces 開發環境配置文件
├── .github/
│   └── workflows/
│       └── trade.yml             # GitHub Actions 核心排程工作流
├── config.py                     # 全域參數設定與環境變數管理
├── strategy.py                   # 核心 H-Model 策略計算與 Gate.io API 交互模組
├── requirements.txt              # Python 依賴套件清單
├── README.md                     # Agent 與開發者說明文件
└── .env.example                  # 本地開發測試環境變數範本
```

三、H-Model 量化策略邏輯重構

Pine Script 腳本轉換為 Python 模組時，需嚴格遵循以下數學模型：

1. 指標計算公式

- Daily BIAS% (日線乖離率)**：取現貨每日收盤價 $P_{spot,D}$ 與 20日簡單移動平均線 $SMA20_D$ ：

$$BIAS\% = ((P_{spot,D} - SMA20_D) / SMA20_D) \times 100$$

- Spread% (現期價差率)**：取永續合約價格 $P_{futures}$ 與現貨價格 P_{spot} ：

$$Spread\% = ((P_{futures} - P_{spot}) / P_{spot}) \times 100$$

2. 進出場訊號規則

訊號類型	觸發條件 (邏輯 AND)	Gate.io 執行動作
Bottom Entry	$BIAS\% < 0$ 且 $Spread\% < 0$	市價建立/加碼做多合約 (Market Buy)
Exit Entry	$BIAS\% > 0$	平掉所有多單持倉 (Market Sell, <code>reduceOnly=True</code>)

四、代碼實現與檔案規格

1. requirements.txt

```
ccxt>=4.0.0
pandas>=2.0.0
requests>=2.28.0
python-dotenv>=1.0.0
```

2. .devcontainer/devcontainer.json

```
{
  "name": "Gate.io Quantitative Trading Container",
  "image": "mcr.microsoft.com/devcontainers/python:1-3.10-bullseye",
  "customizations": {
    "vscode": {
      "extensions": [
        "ms-python.python",
        "GitHub.copilot"
      ]
    }
  },
  "postCreateCommand": "pip install -r requirements.txt"
}
```

3. strategy.py 核心邏輯規範

Agent 在編寫 `strategy.py` 時需實作防護機制：包含 API 異常 retry、Telegram 通知，以及持倉過度加碼防護：

```

import os
import ccxt
import pandas as pd
import requests

GATE_API_KEY = os.getenv('GATE_API_KEY')
GATE_API_SECRET = os.getenv('GATE_API_SECRET')
TG_TOKEN = os.getenv('TELEGRAM_BOT_TOKEN')
TG_CHAT_ID = os.getenv('TELEGRAM_CHAT_ID')

exchange = ccxt.gateio({
    'apiKey': GATE_API_KEY,
    'secret': GATE_API_SECRET,
    'enableRateLimit': True,
    'options': {'defaultType': 'swap'}
})

def run_trading_cycle():
    spot_symbol = 'BTC/USDT'
    futures_symbol = 'BTC/USDT:USDT'

    # 1. Fetch OHLCV
    spot_daily = exchange.fetch_ohlcv(spot_symbol, '1d', limit=30)
    spot_hourly = exchange.fetch_ohlcv(spot_symbol, '1h', limit=2)
    futures_hourly = exchange.fetch_ohlcv(futures_symbol, '1h', limit=2)

    # 2. Calculate Indicators
    df_d = pd.DataFrame(spot_daily, columns=['ts', 'o', 'h', 'l', 'c', 'v'])
    sma20 = df_d['c'].rolling(20).mean().iloc[-1]
    p_spot_d = df_d['c'].iloc[-1]
    bias = ((p_spot_d - sma20) / sma20) * 100

    p_spot = spot_hourly[-1][4]
    p_futures = futures_hourly[-1][4]
    spread_pct = ((p_futures - p_spot) / p_spot) * 100

    print(f"[METRICS] BIAS%: {bias:.4f}%, Spread%: {spread_pct:.4f}%")

    # 3. Position Check & Execution
    long_cond = (bias < 0) and (spread_pct < 0)
    exit_cond = (bias > 0)

    positions = exchange.fetch_positions([futures_symbol])
    current_size = sum([float(p['contracts']) for p in positions if p['side'] == 'long'])

    if long_cond:
        order = exchange.create_market_buy_order(futures_symbol, 1) # 1 Contract
        print(f"[ORDER BOUGHT] Order ID: {order['id']}")
    elif exit_cond and current_size > 0:
        order = exchange.create_market_sell_order(futures_symbol, current_size, params={'reduceOnly':
True})
        print(f"[ORDER CLOSED] Order ID: {order['id']}")

if __name__ == '__main__':
    run_trading_cycle()

```

4. `.github/workflows/trade.yml` 自動化工作流

```
name: Gate.io H-Model Automated Trading Workflow

on:
  schedule:
    - cron: '0 * * * *' # Run every hour
  workflow_dispatch:

jobs:
  execute-strategy:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout Code
        uses: actions/checkout@v3

      - name: Setup Python
        uses: actions/setup-python@v4
        with:
          python-version: '3.10'

      - name: Install Dependencies
        run: |
          python -m pip install --upgrade pip
          pip install -r requirements.txt

      - name: Run Trading Strategy Script
        env:
          GATE_API_KEY: ${ secrets.GATE_API_KEY }
          GATE_API_SECRET: ${ secrets.GATE_API_SECRET }
          TELEGRAM_BOT_TOKEN: ${ secrets.TELEGRAM_BOT_TOKEN }
          TELEGRAM_CHAT_ID: ${ secrets.TELEGRAM_CHAT_ID }
        run: python strategy.py
```

五、AI Agent Codespaces 自動構建指令 (Prompt for Agent)

在 GitHub Codespaces 開啟該 Repo 後，直接輸入以下 Prompt 給 Copilot / Cursor Agent，Agent 將自動完成建置：

給 AI Agent 的執行 Prompt：

```
"Please implement the complete Gate.io H-Model Automated Trading project based on this spec document.
1. Create \`.devcontainer/devcontainer.json\` for Python 3.10 environment.
2. Create \`requirements.txt\` with ccxt, pandas, requests.
3. Create \`strategy.py\` with CCXT gateio client, calculating Daily BIAS% and Spread%, fetching Gate.io Futures position, executing market buy/sell with error handling and Telegram alerts.
4. Create \`.github/workflows/trade.yml\` triggering hourly cron job reading secrets `GATE_API_KEY`, `GATE_API_SECRET`, `TELEGRAM_BOT_TOKEN`, `TELEGRAM_CHAT_ID`.
5. Test \`strategy.py\` locally and ensure code formatting."
```

六、安全性與資產風控規範

- **API Key 權限隔離**：於 Gate.io 申請 API Key 時，僅限勾選 **Futures (永續合約)** - **讀取/寫入**，嚴禁勾選 **提現 (Withdraw)** 權限。
- **GitHub Secrets 機密保護**：所有 API Key 必須儲存於 GitHub Secrets，不得直接寫死 (Hardcode) 在代碼中。
- **倉位控管與槓桿防護**：策略於 Gate.io 執行時建議開立低槓桿 (1x ~ 3x)，避免極端行情引發高額回撤或清算。